Chapter 3

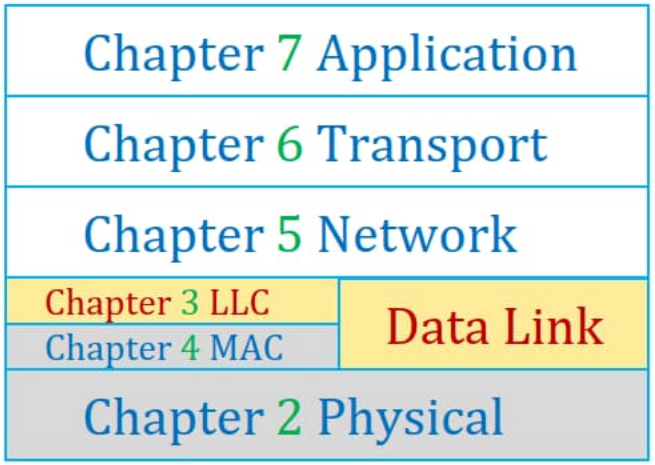
THE DATA LINK LAYER

LOGICAL LINK CONTROL SUBLAYER

Contents

DLL: Data Link Layer

- LLC: Logical Link Control
- MAC: Medium Access Control



Contents

3.1 Data Link Layer Design Issues

- 3.1.1 Services Provided to the Network Layer
- 3.1.2 Framing
- 3.1.3 Error Control
- 3.1.4 Flow Control

frame: 帧
framing: 成帧

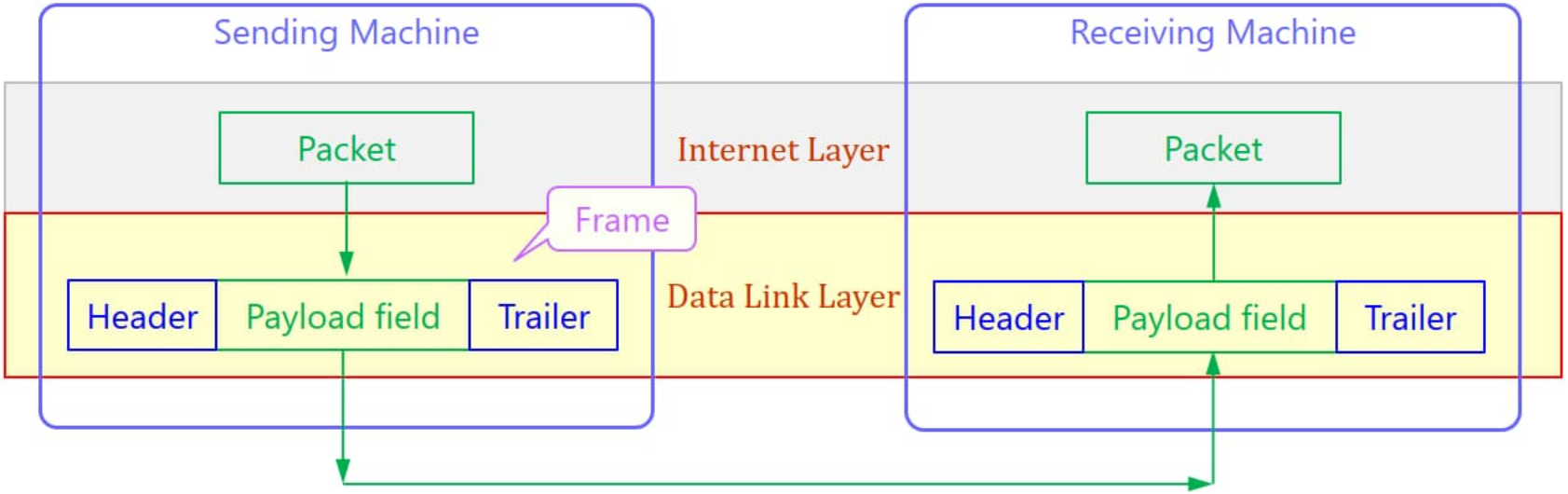
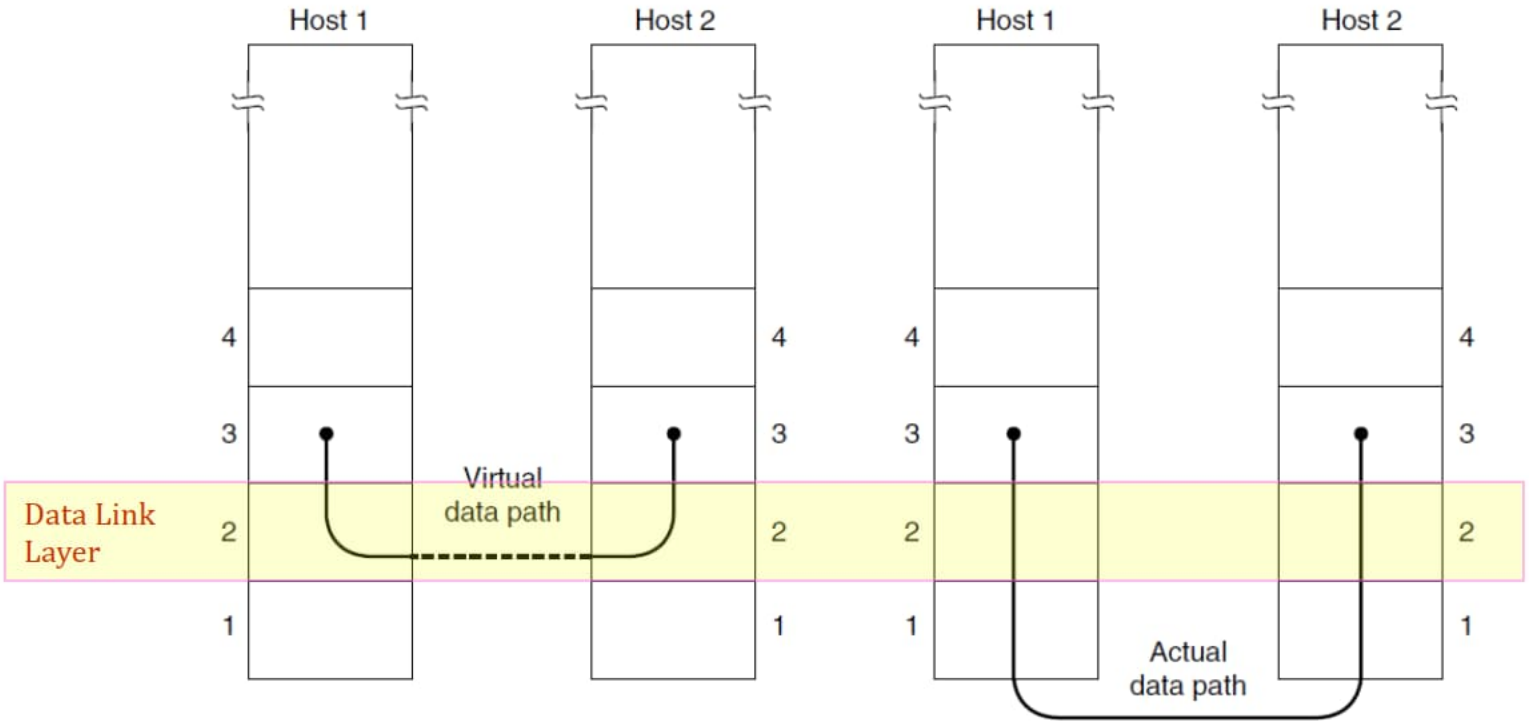


Figure 3-1. Relationship between packets and frames.

3.1.1 Services Provided to the Network Layer

- 1. Unacknowledged connectionless service.
- 2. Acknowledged connectionless service.
- 3. Acknowledged connection-oriented service.



(a) Virtual communication. (b) Actual communication.

Figure 3-2.

3.1.2 Framing

- for error controlling
  - Breaking up the packet into frames,
  - compute a short token called a checksum for each frame.
- framing methods
  - Byte count. 字符计数法
  - Flag bytes with byte stuffing. 带字节填充的定界符法
  - Flag bits with bit stuffing. 带比特填充的定界符法
  - Physical layer coding violations. 物理层编码违例

Byte count

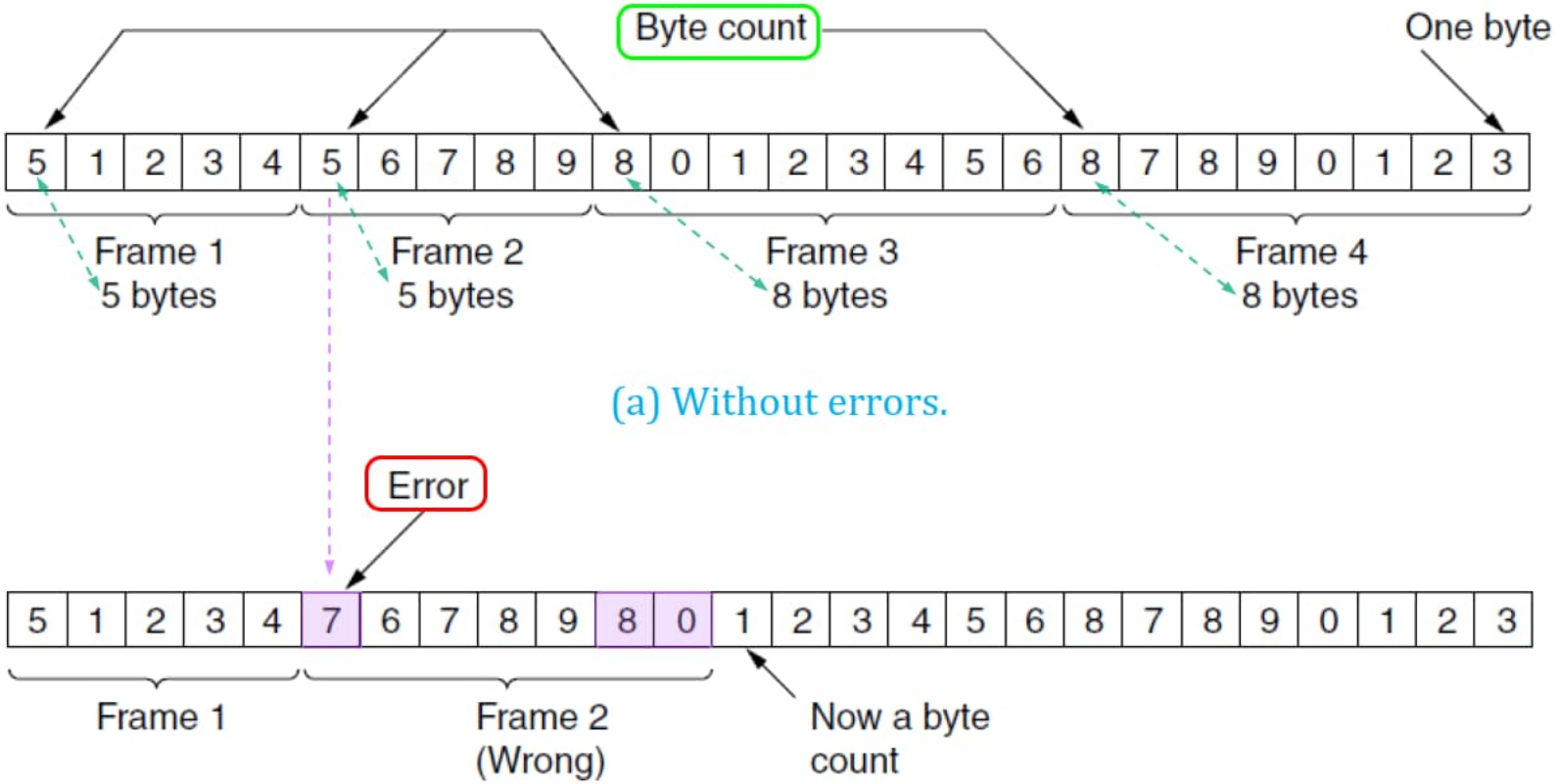


Figure 3-3. A byte stream.

Flag bytes with byte stuffing

Figure 3-4. (a) A frame delimited by flag bytes.

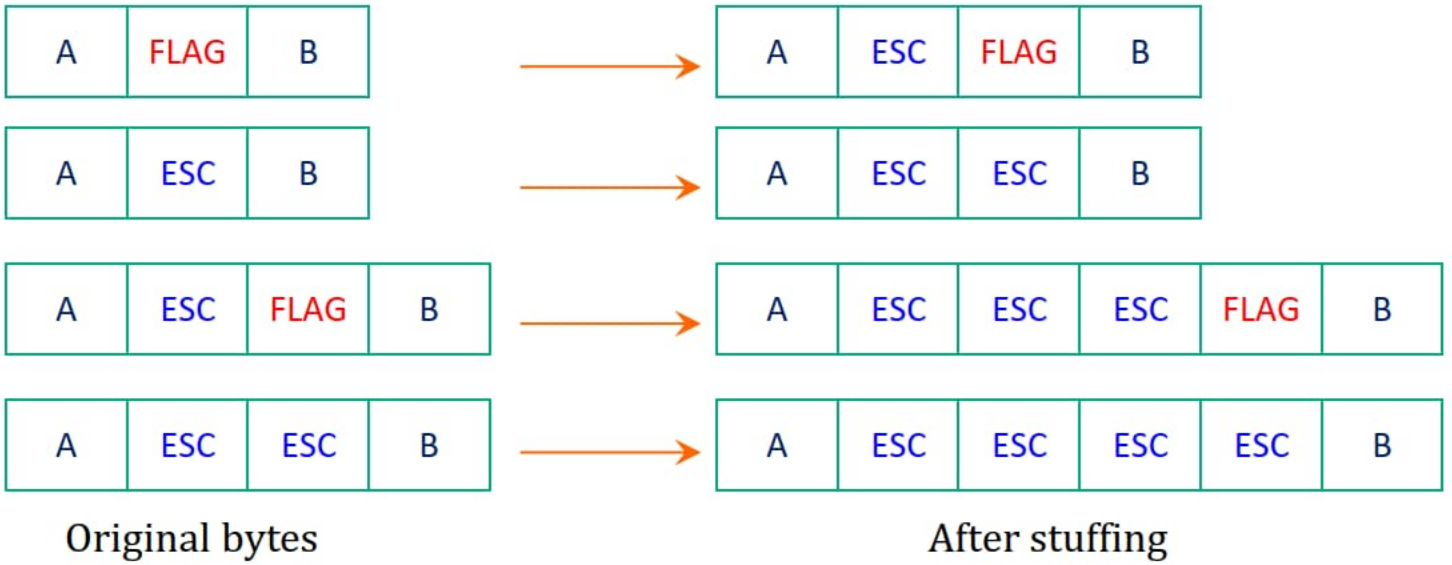


Figure 3-4. (b) Four examples of byte sequences before and after byte stuffing.

Flag bits with bit stuffing

- flag byte
  - Each frame begins and ends with a special bit pattern, 01111110 or 0x7E in hexadecimal.
  - whenever the sender's data link layer encounters 5 consecutive 1 (one) in the data, it automatically stuffs 0 into the outgoing stream.
  - used in HDLC (Highlevel Data Link Control) protocol

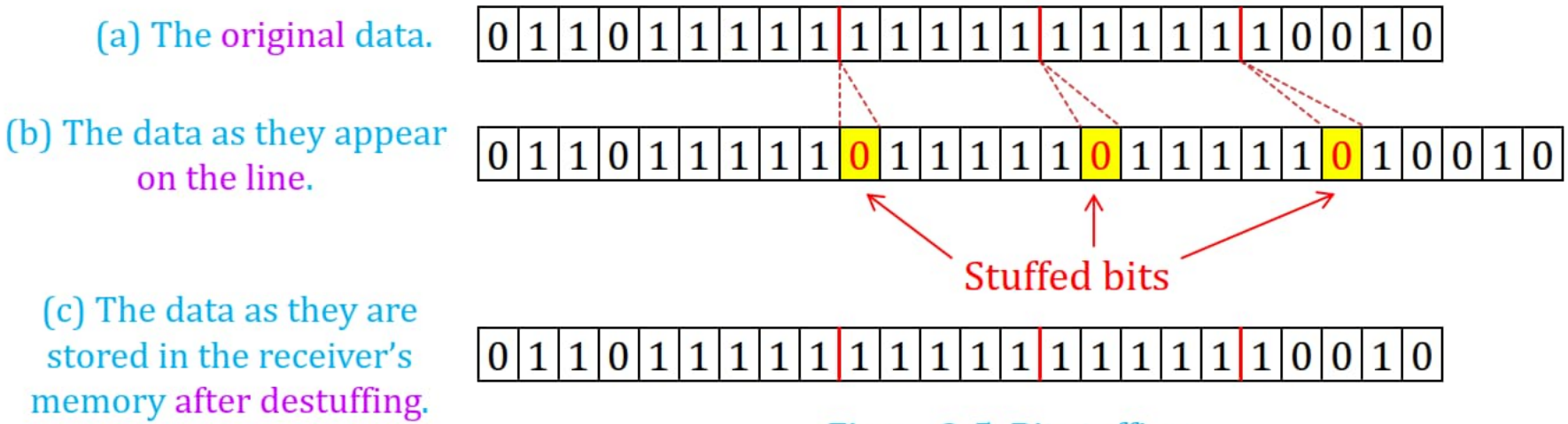


Figure 3-5. Bit stuffing.

Physical layer coding violations

- in the 4B/5B line code
  - 4 data bits are mapped to 5 signal bits
  - 16 out of the 32 signal possibilities are not used.
- for Ethernet and 802.11
  - a frame begin with a well-defined pattern called a preamble
- Manchester encoding or Differential Manchester encoding
  - high-low pair ⌐ /low-high pair ⌐ represent for 1/0
  - so, high-high / low-low maybe used for flag.

3.1.3 Error Control

- Error problems
  - incorrect
  - lost
  - out of order
  - repeatedly delivery
- solution schemes
  - error detect/correct
  - acknowledgement
  - timer
  - sequence number

3.1.4 Flow Control

- flow problem
  - a sender may transmit frames faster than the receiver can accept them.
- Two approaches are commonly used
  - feedback-based flow control
    - the receiver sends back information to the sender giving it permission to send more data, or at least telling the sender how the receiver is doing.
  - rate-based flow control
    - the protocol has a built-in mechanism that limits the rate at which senders may transmit data, without using feedback from the receiver.

3.2 Error Detection and Correction

- two basic strategies for dealing with errors.
  - error-correcting codes, 纠错码
    - include enough redundant information to enable the receiver to deduce what the transmitted data must have been.
    - often referred to as **FEC** (Forward Error Correction).
    - used on noisy channels, such as wireless links.
  - error-detecting codes, 检错码
    - include only enough redundancy to allow the receiver to deduce that an error has occurred (but not which error) and have it request a retransmission. (ARQ: Automatic Repeat-reQuest)
    - used on highly reliable channels, such as fiber.
  - Both add redundant information to the data that is sent.

3.2.1 Error-Correcting Codes

- four different error-correcting codes
  - 1. Hamming codes. 海明码
  - 2. Binary convolutional codes. 二进制卷积码
  - 3. Reed-Solomon codes. 里德所罗门码
  - 4. Low-Density Parity Check codes. 低密度奇偶校验码
- n-bit codeword, 码字
  - An n-bit unit containing data and check bits.
  - A frame consists of m message (i.e., data) bits and r redundant (i.e. check) bits. Let the total length of a block be n (i.e., $n = m + r$).
  - We will describe this as an (n, m) code.
- code rate, 编码速率，码率
  - is the fraction of the codeword that carries information that is not redundant, or m/n.

Hamming distance

- Hamming distance 海明距离
  - the Hamming distance between two codewords
    - 两个码字之间的海明距离
    - The number of bit positions in which two codewords differ
      - the Hamming distance between 10001001 and 10110001 is 3.
  - the Hamming distance of the complete code
    - 编码方案的海明距离
    - the two codewords with the smallest Hamming distance.

error-detect / correct

- Hamming distance & the error-detecting and error-correcting
  - To reliably detect d errors, you need a distance $(d + 1)$ code.
    - Parity check codes
      - 10110101 → 101101011 (even parity check)
      - 10110001 → 101100010
  - to correct d errors, you need a distance $(2d + 1)$ code.
    - a code with only four valid codewords:
      - 0000000000, 0000011111,
      - 1111100000, 1111111111
    - This code has a distance of 5, which means that it can
      - correct double errors, or
      - detect quadruple errors.

the number of check bits

- the number of check bits
  - allow all single errors to be corrected
  - an n-bits code with m message bits and r check bits
    - $n = m + r$
    - Number of legal messages: 2^m
  - we must have: $(n + 1) \cdot 2^m \leq 2^n$.
    - Each legal message has n illegal codewords at a distance of 1 from it,
    - and 1 legal codeword.
  - we got: $(m + r + 1) \leq 2^{n-m} = 2^r$, Using $n = m + r$

m	1	2-4	5-11	12-26	27-57	...
r	2	3	4	5	6	...

Given m, this puts a lower limit on the number of check bits needed to correct single errors.

Hamming codes

- the bits of the codeword are numbered consecutively, from left to right
- The bits that are powers of 2 (1, 2, 4, 8, 16, etc.) are check bits.
- The rest (3, 5, 6, 7, 9, etc.) are filled up with the m data bits.
- rewrite data bit k as a sum of powers of 2
  - $11 = 1 + 2 + 8$, and
  - $29 = 1 + 4 + 8 + 16$
- A bit is checked by just those check bits occurring in its expansion
  - bit 11 is checked by bits 1, 2, and 8
  - bit 29 is checked by bits 1, 4, 8, and 16
- the check bits are computed for even parity sums for a message

Examples of Hamming Codes

Char	ASCII	Check bits bit: 12345678901
H	1001000	00110010000
a	1100001	10110001001
m	1101101	11101010101
m	1101101	11101010101
i	1101001	01101011001
n	1101110	01101010110
g	1100111	01111001111
	0100000	10011000000
c	1100011	11111000011
o	1101111	10101011111
d	1100100	11111001100
e	1100101	00111000101

interleaved code, 交织码

$m = 7, r = 4, n = m + r = 11$

even parity sums

data bits:

3=1+2, 5=1+4

6=2+4, 7=1+2+4

9=1+8, 10=2+8

11=1+2+8

Check bits:

$1 \in (3, 5, 7, 9, 11)$

$2 \in (3, 6, 7, 10, 11)$

$4 \in (5, 6, 7)$

$8 \in (9, 10, 11)$

it can correct single errors (or detect double errors).
The set of check results forms the error syndrome that is used to pinpoint and correct the error.

3.2.2 Error-Detecting Codes

- three different error-detecting codes
  - Parity. 奇偶校验
    - even parity, 偶校验
    - odd parity, 奇校验
  - Checksums. 校验和
    - longitudinal parity check, 纵向奇偶校验
    - Modular 2^{16} sum, 模和
      - one's complement arithmetic, 补码运算
  - Cyclic Redundancy Checks (CRCs). 循环冗余校验

Interleaving of parity bits

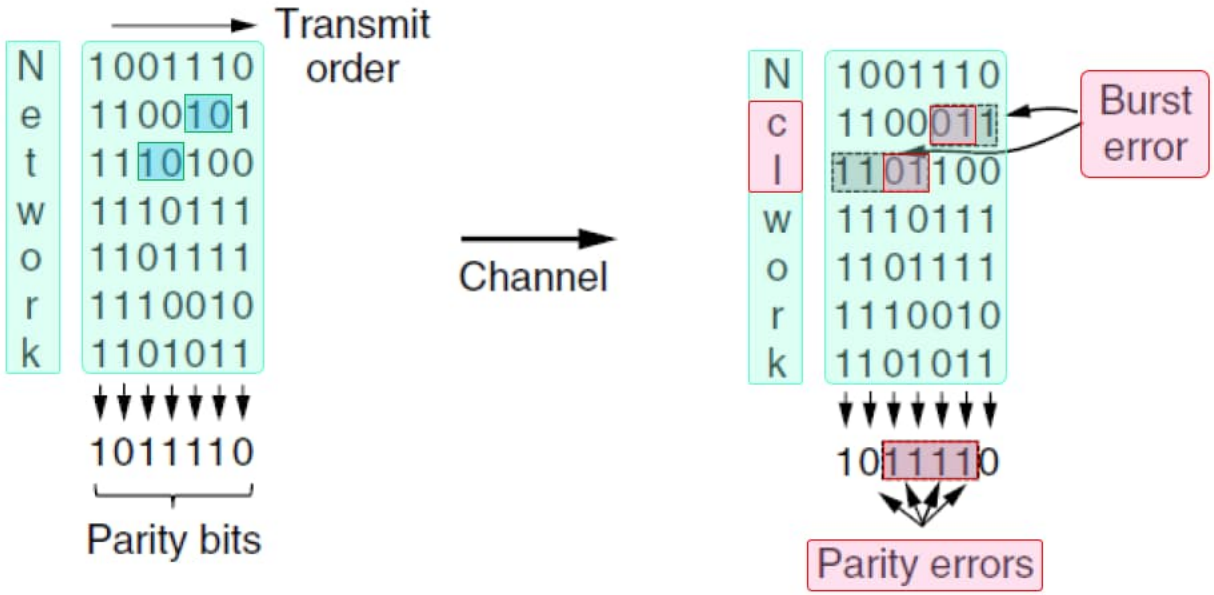


Figure 3-8. Interleaving of parity bits to detect a burst error.

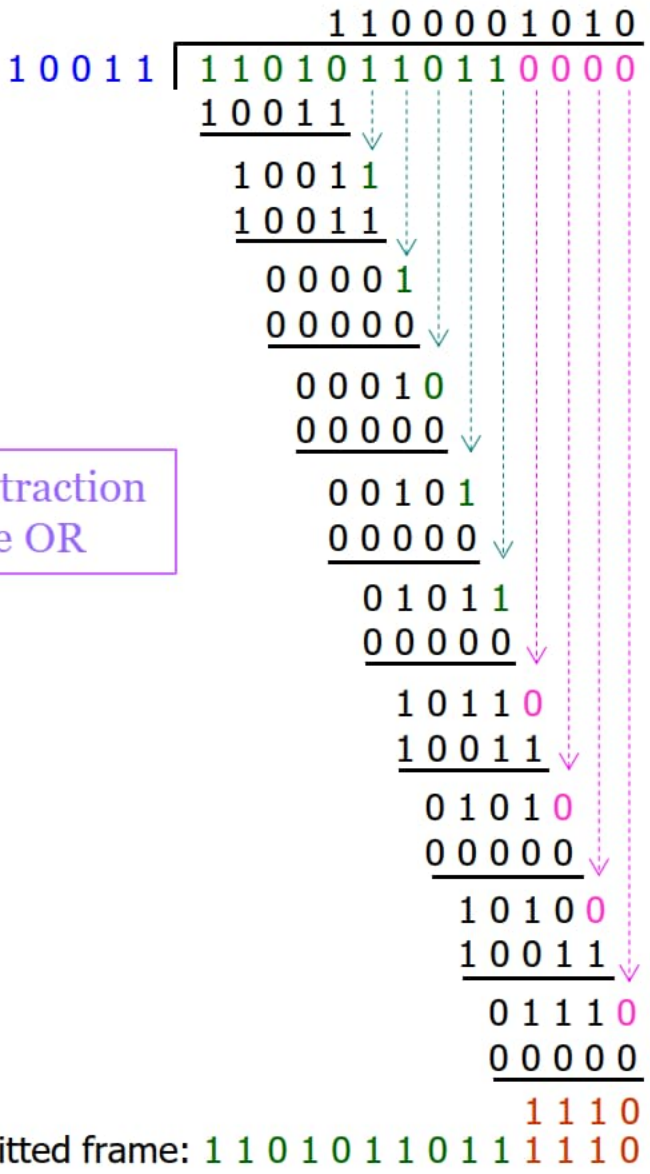
Cyclic Redundancy Check

- 110001 has 6 bits and thus represents a six-term polynomial
  - $1\,x^5 + 1\,x^4 + 0\,x^3 + 0\,x^2 + 0\,x^1 + 1\,x^0$
- Sending
  - Message frame polynomial is $M(x)$, (m) bits
  - Append r zero bits to the low-order end of the frame so it now contains $(m + r)$ bits and corresponds to the polynomial $x^r \cdot M(x)$
  - divided by $G(x)$, $(r+1)$ bits
  - remainder is $R(x)$
  - $T(x) = [x^r \cdot M(x) + R(x)]$ is the checksummed frame to be transmitted
- Receiving

$$\frac{x^r \cdot M(x) + R(x)}{G(x)} = \frac{x^r \cdot M(x) - R(x)}{G(x)} = 0$$

modulo 2 subtraction = exclusive OR

Frame : 1 1 0 1 0 1 1 0 1 1
Generator : 1 0 0 1 1
Message after 4 zero bits are appended: 1 1 0 1 0 1 1 0 1 1 0 0 0 0



← Remainder

modulo 2 subtraction
= exclusive OR

Transmitted frame: 1 1 0 1 0 1 1 0 1 1 1 1 1 0

international standards

- IEEE 802
  - CRC-12:
    - $x^{12}+x^{11}+x^3+x^2+x+1$
  - CRC-16:
    - $x^{16}+x^{15}+x^2+1$
  - CRC-CCITT:
    - $x^{16}+x^{15}+x^5+1$
  - CRC-32:
    - $x^{32}+x^{26}+x^{23}+x^{22}+x^{16}+x^{12}+x^{11}+x^{10}+x^8+x^7+x^5+x^4+x^2+x+1$

3.3 Elementary Data Link Protocols

file protocol.h

```
#define MAX_PKT 1024                                        /* determines packet size in bytes */
typedef enum {false, true} boolean;                         /* boolean type */
typedef unsigned int seq_nr;                                /* sequence or ack numbers */
typedef struct {unsigned char data[MAX_PKT];} packet;       /* packet definition */
typedef enum {data, ack, nak} frame_kind;                   /* frame kind definition */

typedef struct {                                            /* frames are transported in this layer */
        frame_kind kind;                                    /* what kind of frame is it? */
        seq_nr seq;                                         /* sequence number */
        seq_nr ack;                                         /* acknowledgement number */
        packet info;                                        /* the network layer packet */
} frame;
```

```
void wait_for_event(event_type *event);
void from_network_layer(packet *p);
void to_network_layer(packet *p);
void from_physical_layer(frame *r);
void to_physical_layer(frame *s);
void start_timer(seq_nr k);
void stop_timer(seq_nr k);
void start_ack_timer(void);
void stop_ack_timer(void);
void enable_network_layer(void);
void disable_network_layer(void);
#define inc(k) if (k < MAX_SEQ) k = k + 1; else k = 0
```

Figure 3-11. Some definitions needed in the protocols to follow. These definitions are located in the file protocol.h.

3.3.1 A Utopian Simplex Protocol

```
typedef enum {frame_arrival} event_type;
#include "protocol.h"
void sender1(void)
{   frame s;                          /* buffer for an outbound frame */
    packet buffer;                    /* buffer for an outbound packet */
    while (true) {
        from_network_layer(&buffer);  /* go get something to send */
        s.info = buffer;              /* copy it into s for transmission */
        to_physical_layer(&s);        /* send it on its way */
    }
}

void receiver1(void)
{   frame r;
    event_type event;                 /* filled in by wait, but not used here */
    while (true) {
        wait_for_event(&event);       /* only possibility is frame arrival */
        from_physical_layer(&r);      /* go get the inbound frame */
        to_network_layer(&r.info);    /* pass the data to the network layer */
    }
}
```

3.3.2 A Simplex Stop-and-Wait Protocol for an Error-Free Channel

```
typedef enum {frame_arrival} event_type;
#include "protocol.h"
void sender2(void)
{   frame s;                          /* buffer for an outbound frame */
    packet buffer;                    /* buffer for an outbound packet */
    event_type event;                 /* frame arrival is the only possibility */
    while (true) {
        from_network layer(&buffer);  /* go get something to send */
        s.info = buffer;              /* copy it into s for transmission */
        to_physical_layer(&s);        /* bye-bye little frame */
        wait_for_event(&event);       /* do not proceed until given the go ahead */
    }
}
void receiver2(void)
{   frame r, s;                       /* buffers for frames */
    event_type event;                 /* frame_arrival is the only possibility */
    while (true) {
        wait_for_event(&event);       /* only possibility is frame_arrival */
        from_physical_layer(&r);      /* go get the inbound frame */
        to_network_layer(&r.info);    /* pass the data to the network layer */
        to_physical_layer(&s);        /* send a dummy frame to awaken sender */
    }
}
```

3.3.3 A Simplex Stop-and-Wait Protocol for a Noisy Channel

```
#define MAX_SEQ 1                                   /* must be 1 for protocol 3 */
typedef enum {frame_arrival, cksum_err, timeout} event_type;
#include "protocol.h"
void sender3(void)
{   seq_nr next_frame_to_send;                      /* seq number of next outgoing frame */
    frame s;                                        /* scratch variable */
    packet buffer;                                  /* buffer for an outbound packet */
    event_type event;
    next_frame_to_send = 0;                         /* initialize outbound sequence numbers */
    from_network_layer(&buffer);                    /* fetch first packet */
    while (true) {
        s.info = buffer;                            /* construct a frame for transmission */
        s.seq = next_frame_to_send;                 /* insert sequence number in frame */
        to_physical_layer(&s);                      /* send it on its way */
        start_timer(s.seq);                         /* if answer takes too long, time out */
        wait_for_event(&event);                     /* frame_arrival, cksum_err, timeout */
        if (event == frame_arrival) {
            from_physical_layer(&s);                /* get the acknowledgement */
            if (s.ack == next_frame_to_send) {
                stop_timer(s.ack);                  /* turn the timer off */
                from_network_layer(&buffer);        /* get the next one to send */
                inc(next_frame_to_send);            /* invert next_frame_to_send */
}   }   }   }
```

```
#define MAX_SEQ 1                                   /* must be 1 for protocol 3 */
typedef enum {frame_arrival, cksum_err, timeout} event_type;
#include "protocol.h“
void receiver3(void)
{   seq_nr frame_expected;
    frame r, s;
    event_type event;
    frame_expected = 0;
    while (true) {
        wait_for_event(&event);                     /* possibilities: frame_arrival, cksum_err */
        if (event == frame_arrival) {               /* a valid frame has arrived */
            from_physical_layer(&r);                /* go get the newly arrived frame */
            if (r.seq == frame_expected) {          /* this is what we have been waiting for */
                to_network_layer(&r.info);          /* pass the data to the network layer */
                inc(frame_expected);                /* next time expect the other sequence nr */
            }
            s.ack = 1 – frame_expected;             /* tell which frame is being acked */
            to_physical_layer(&s);                  /* send acknowledgement */
        }
    }
}
```

3.4 Sliding Window Protocols

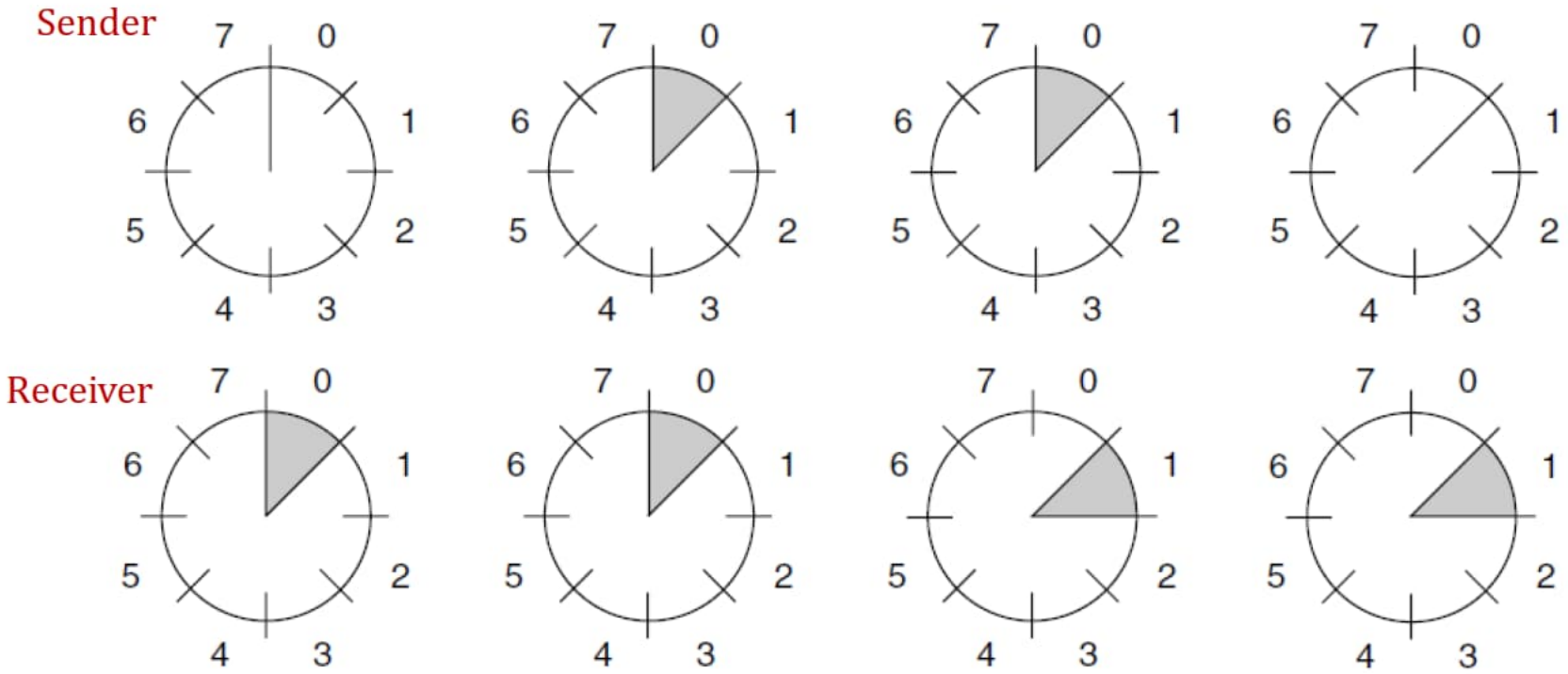


Figure 3-15. A sliding window of size 1, with a 3-bit sequence number.

3.4.1 A One-Bit Sliding Window Protocol

```
/* Protocol 4 (Sliding window) is bidirectional. */

#define MAX_SEQ 1                                   /* must be 1 for protocol 4 */
typedef enum {frame_arrival, cksum_err, timeout} event_type;
#include "protocol.h"

void protocol4 (void)
{   seq_nr next_frame_to_send;                      /* 0 or 1 only */
    seq_nr frame_expected;                          /* 0 or 1 only */
    frame r, s;                                     /* scratch variables */
    packet buffer;                                  /* current packet being sent */
    event_type event;
    next_frame_to_send = 0;                         /* next frame on the outbound stream */
    frame_expected = 0;                             /* frame expected next */
    from_network_layer(&buffer);                    /* fetch a packet from the network layer */
    s.info = buffer;                                /* prepare to send the initial frame */
    s.seq = next_frame_to_send;                     /* insert sequence number into frame */
    s.ack = 1 – frame_expected;                     /* piggybacked ack */

    to_physical_layer(&s);                          /* transmit the frame */
    start_timer(s.seq);                             /* start the timer running */
```

A One-bit sliding window protocol

```
while (true) {
    wait_for_event(&event);                    /* frame_arrival, cksum_err, or timeout */
    if (event == frame_arrival) {              /* a frame has arrived undamaged */
        from_physical_layer(&r);               /* go get it */
        if (r.seq == frame_expected) {         /* handle inbound frame stream */
            to_network_layer(&r.info);         /* pass packet to network layer */
            inc(frame_expected);               /* invert seq number expected next */
        }
        if (r.ack == next_frame_to_send) {     /* handle outbound frame stream */
            stop_timer(r.ack);                 /* turn the timer off */
            from_network_layer(&buffer);       /* fetch new pkt from network layer */
            inc(next_frame_to_send);           /* invert sender's sequence number */
        }
    }
    s.info = buffer;                           /* construct outbound frame */
    s.seq = next_frame_to_send;                /* insert sequence number into it */
    s.ack = 1 – frame_expected;                /* seq number of last received frame */
    to_physical_layer(&s);                     /* transmit a frame */
    start_timer(s.seq);                        /* start the timer running */
}                                              /* end of while */
}                                              /* end of protocol 4 */
```

Two scenarios for protocol 4

The notation is (seq, ack, packet_number).
An up-arrow indicates where a network layer accepts a packet.

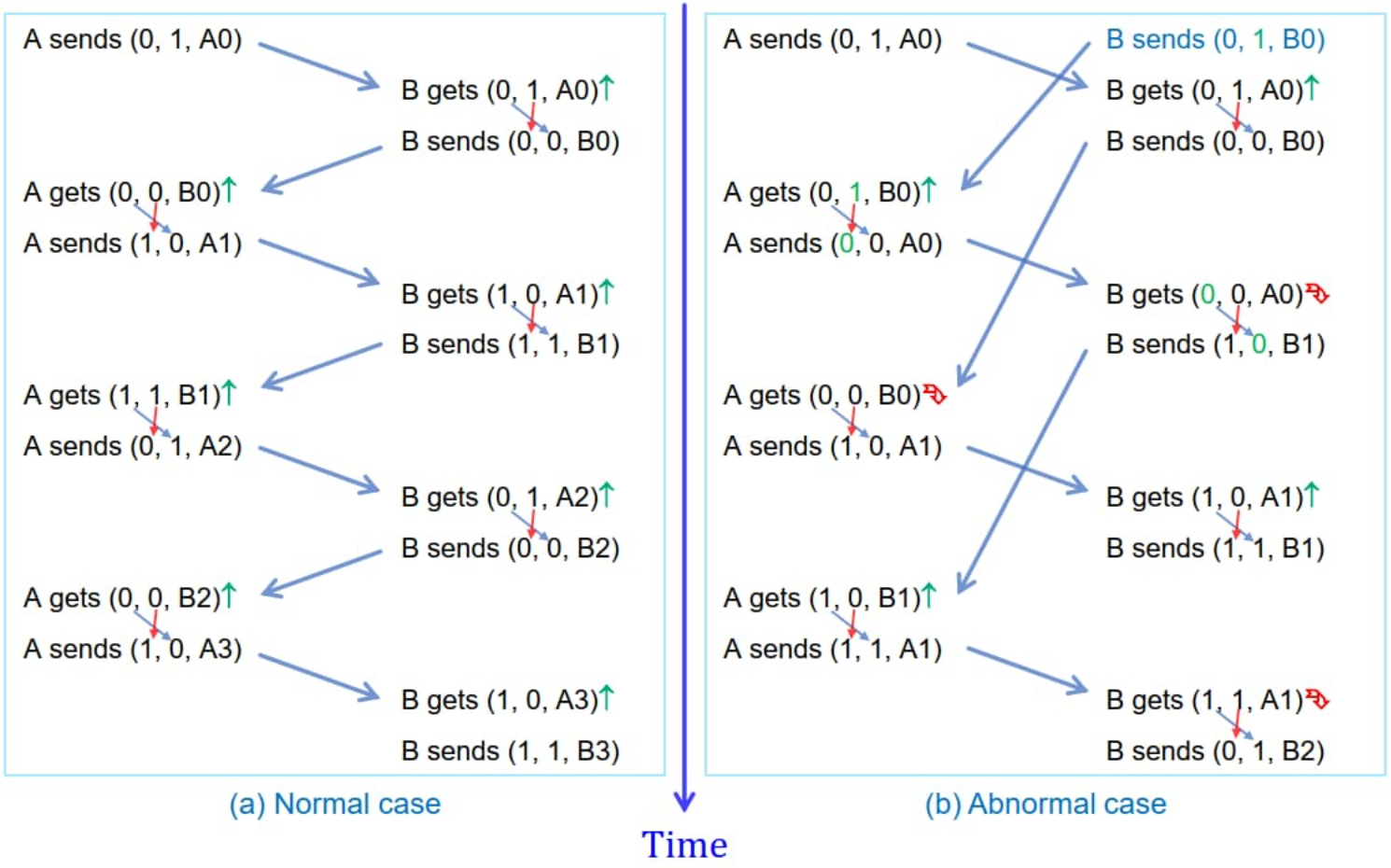


Figure 3-17. Two scenarios for protocol 4

3.4.2 A Protocol Using Go-Back-N

- transmission time: S/b
- frame arrived: S/b + d
- ack to come back: S/b + d + d

S: frame-size
b: bandwidth
d: delay

$$\text{link utilization} \le \frac{\frac{S}{b}}{\frac{S}{b}+d+d} = \frac{1}{1+2\times\frac{b\cdot d}{S}}$$

here: $b \cdot d$ means bandwidth-delay product of the link.

pipelining: keeping multiple (w) frames in flight.

$$\text{link utilization} \le \frac{w}{1+2\times\frac{b\cdot d}{S}}, \quad \text{for window size } w \text{ .}$$

Go-Back-N

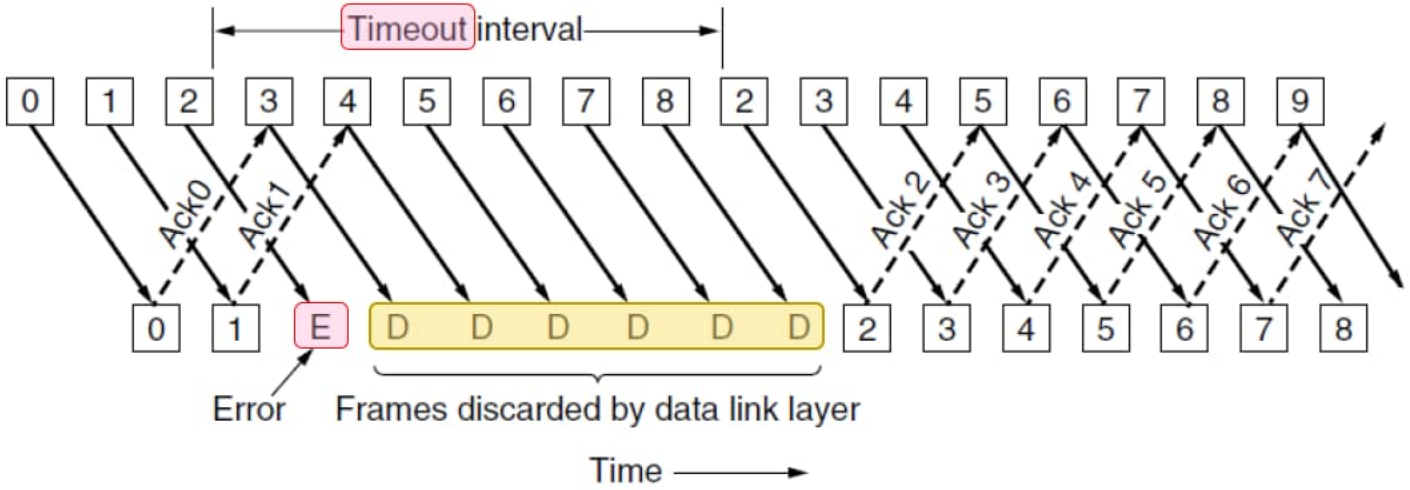


Figure 3-18. Pipelining and error recovery. Effect of an error when

(a) receiver's window size is 1 and

/* Protocol 5 (Go-back-n) allows multiple outstanding frames. The sender may transmit up to MAX SEQ frames without waiting for an ack. In addition, unlike in the previous protocols, the network layer is not assumed to have a new packet all the time. Instead, the network layer causes a network layer ready event when there is a packet to send. */

```
#define MAX SEQ 7
typedef enum {frame_arrival, cksum_err, timeout, network_layer_ready} event_type;
#include "protocol.h"

static boolean between(seq_nr a, seq_nr b, seq_nr c)
{   /* Return true if a <= b < c circularly; false otherwise. */
    if (((a <= b) && (b < c)) || ((c < a) && (a <= b)) || ((b < c) && (c < a)))
        return(true);
    else
        return(false);
}

static void send_data(seq_nr frame_nr, seq_nr frame_expected, packet buffer[ ])
{   /* Construct and send a data frame. */
    frame s;                                        /* scratch variable */
    s.info = buffer[frame_nr];                      /* insert packet into frame */
    s.seq = frame_nr;                               /* insert sequence number into frame */
    s.ack = (frame_expected + MAX_SEQ) % (MAX_SEQ + 1);        /* piggyback ack */
    to_physical_layer(&s);                          /* transmit the frame */
    start_timer(frame_nr);                          /* start the timer running */
}
```

```
while (true) {
    wait_for_event(&event);                 /* four possibilities: see event type above */
    switch(event) {
      case network_layer_ready:             /* the network layer has a packet to send */
        /* Accept, save, and transmit a new frame. */
        from_network_layer(&buffer[next_frame_to_send]);   /* fetch new packet */
        nbuffered = nbuffered + 1;          /* expand the sender's window */
        send_data(next_frame_to_send, frame_expected, buffer);/* transmit the frame */
        inc(next_frame_to_send);            /* advance sender's upper window edge */
        break;

      case frame_arrival:                   /* a data or control frame has arrived */
        from_physical_layer(&r);            /* get incoming frame from physical layer */
        if (r.seq == frame_expected) {
            /* Frames are accepted only in order. */
            to_network_layer(&r.info);      /* pass packet to network layer */
            inc(frame_expected);            /* advance lower edge of receiver's window */
        }
        /* Ack n implies n − 1, n − 2, etc. Check for this. */
        while (between(ack_expected, r.ack, next_frame_to_send)) {
            /* Handle piggybacked ack. */
            nbuffered = nbuffered − 1;      /* one frame fewer buffered */
            stop_timer(ack_expected);       /* frame arrived intact; stop timer */
            inc(ack_expected);              /* contract sender's window */
        }
        break;
```

```
            case cksum_err: break;              /* just ignore bad frames */

            case timeout:                       /* trouble; retransmit all outstanding frames */
               next_frame_to_send = ack_expected; /* start retransmitting here */
               for (i = 1; i <= nbuffered; i++) {
                   send_data(next_frame_to_send, frame_expected, buffer);/* resend frame */
                   inc(next_frame_to_send);     /* prepare to send the next one */
               }
            }

         if (nbuffered < MAX_SEQ)
            enable_network_layer();
         else
            disable_network_layer();

      }     /* end of switch case */

   }  /* end of protocol 5 */
```

Figure 3-19. A sliding window protocol using go-back-n.

3.4.3 A Protocol Using Selective Repeat

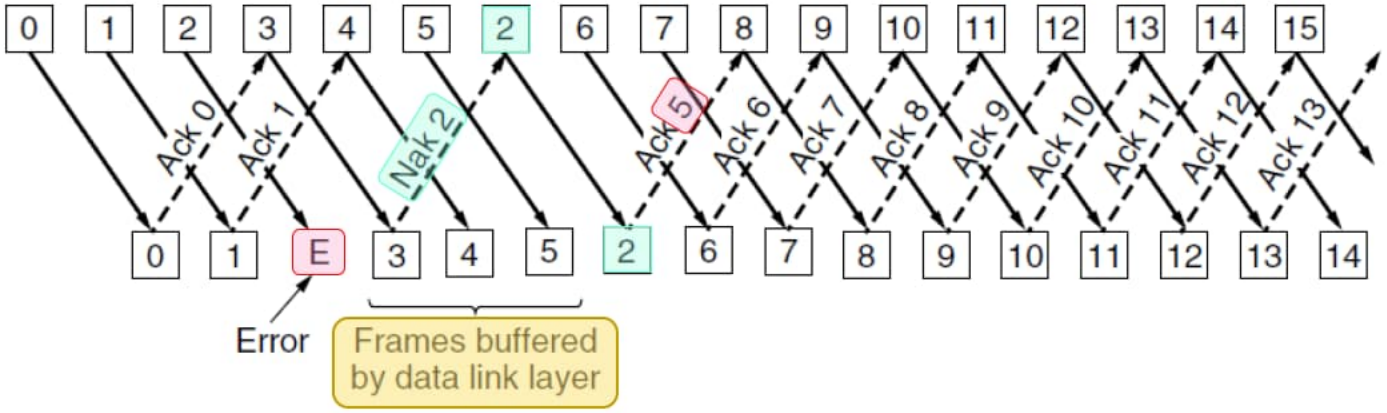


Figure 3-18. Pipelining and error recovery. Effect of an error when

(b) receiver's window size is large.

/* Protocol 6 (Selective repeat) (ignored) */

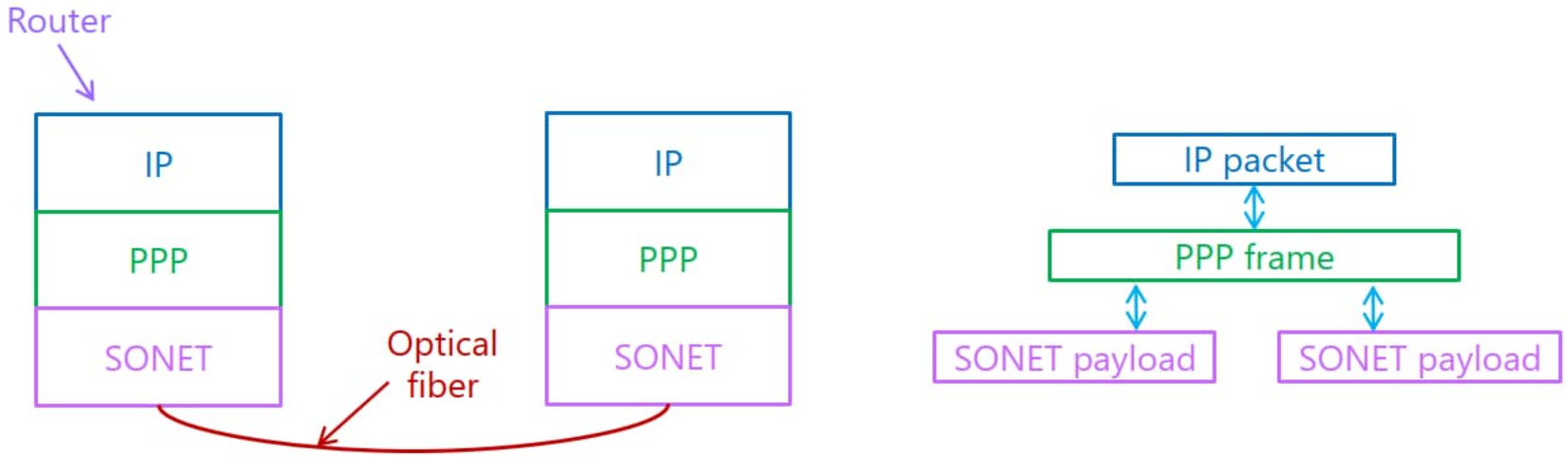
3.5 Example Data Link Protocols

- PPP (Point-to-Point Protocol)
  - is used to send packets over point-to-point links.
  - is defined in RFC 1661 and further elaborated in RFC 1662 and other RFCs.
  - SONET and ADSL links both apply PPP, but in different ways.

- over SONET optical fiber links
  - in wide-area networks

- for ADSL links
  - running on the local loop of the telephone network at the edge of the Internet

3.5.1 Packet over SONET



(a) A protocol stack.

(b) Frame relationships.

Figure 3-23. Packet over SONET.

PPP frame format

Protocol 0021	IP datagram
Protocol C021	LCP
Protocol 8021	NCP

Figure 3-24. The PPP full frame format for unnumbered mode operation.

可以通过LCP协商，移去 Addr 和 Control 两个字段。

PPP State diagram

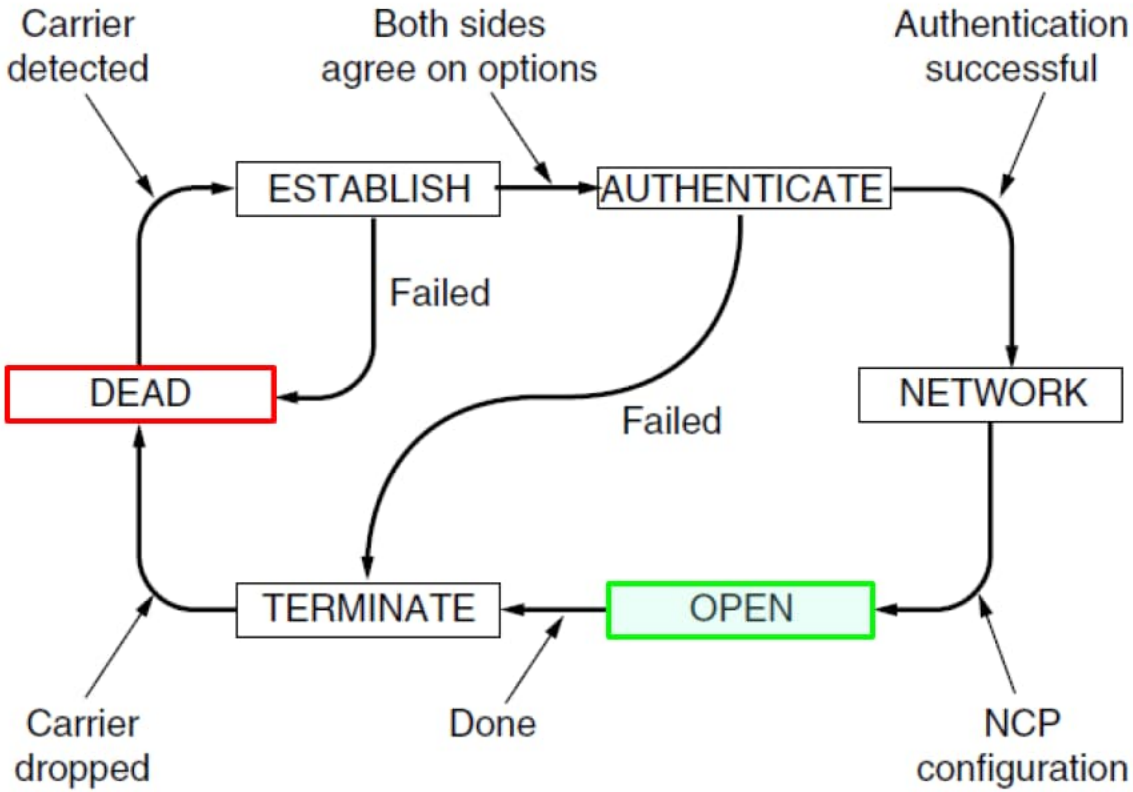


Figure 3-25. State diagram for bringing a PPP link up and down.

3.5.2 ADSL

- ADSL (Asymmetric Digital Subscriber Loop)

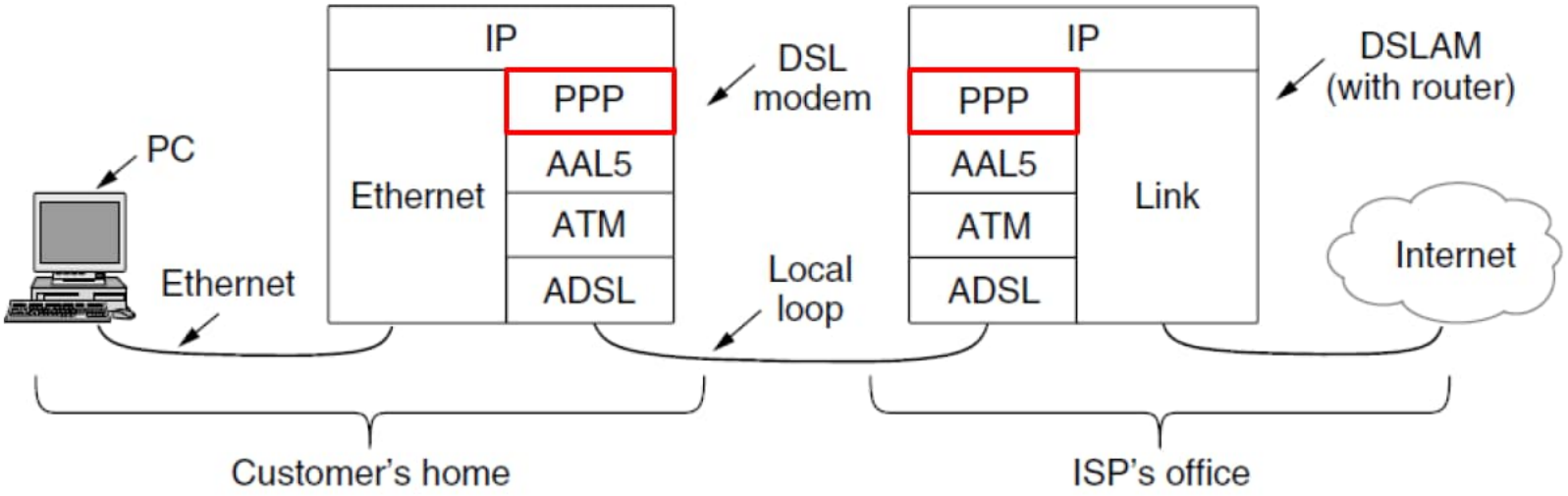


Figure 3-26. ADSL protocol stacks.

ATM: Asynchronous Transfer Mode

AAL5: ATM Adaptation Layer type 5

Summary

- The task of the data link layer is to
  - convert the raw bit stream offered by the physical layer
    - into a stream of frames for use by the network layer.
- Various framing methods are used, including
  - byte count, byte stuffing, and bit stuffing.
- Data link protocols can provide
  - error control: error correction & error detection.
  - flow control: sliding window mechanism.
- provide a reliable link layer using
  - ARQ
    - acknowledgements
    - retransmissions

Logical Link Control